Reversible Agency: A Permissioned, Provenance-Carrying Action Layer for LLM Operating Systems

Youssef Hariri
DeeWaanAI
github.com/DeeWaanAI

Draft — October 2026

Abstract

LLM agents can now call tools, move files, and send messages. The literature has made them good at *acting*. It has not made them accountable for it. We present Khatt, the action layer of the DeeWaanAI OS stack, built on one claim: every agent action must be a **bound record** — what was asked, what was done, on whose authority, with what data — and every action must be **reversible** or explicitly marked irreversible before it runs. In Khatt, an action cannot execute unless it produces this record; permission scopes are checked by deterministic code at commit time, never by model judgment; and an executable undo recipe is captured before execution, not reconstructed after. Over a frozen battery of 41 actions, Khatt restored state byte-for-byte for 100% of undoable actions, correctly gated both irreversible actions (email send, trash purge) behind explicit recorded confirmation, caught 15/15 injected permission and precondition defects before execution with zero side effects, and produced records from which every action was fully re-narratable, with tampering to the action log detected by a hash chain. We also report what the system provably cannot do: irreversible actions cannot be undone, only gated; undo conflicts after external modification are detected and refused, not resolved; and the gate constrains normalized actions, not the fidelity of intent parsing itself.

1 Introduction

An LLM operating system becomes useful — and dangerous — at the point where it stops rendering and starts acting. Moving a file, booking a meeting, or sending an email differs from generating a view in one decisive way: the world changes, and the change may not come back.

Current agent systems treat an action as a transient function call. Arguments go in, a result comes out, and benchmarks score whether the task succeeded. What is missing is the receipt: a durable record of what was asked, who authorized it, what data it touched, and how to take it back. Operating systems solved this problem for programs decades ago with permissions, capabilities, and audit logs. The emerging LLM-OS literature reuses the vocabulary — kernels, processes, scheduling — while omitting the enforcement and accountability core.

Khatt is the action layer of DeeWaanAI OS, a layered architecture in which each layer is simultaneously a software module and an academic paper. Layer 1, Qafiya, provides a provenance-first data substrate in which every record carries its origin. Khatt extends that discipline from *data* to *action*. Its contribution is not a new tool-calling mechanism, a new permission filter, or a new logging format. It is the integration of three properties into one artifact:

1. **Permissioned**: no action executes outside an explicit, structured grant, checked by deterministic code at commit time.

2. **Recorded**: every action — including refused and failed ones — is stored as a provenance-carrying Qafiya record in a hash-chained log.
3. **Reversible**: an executable undo recipe is captured before execution, and every action is classified `EXACT`, `COMPENSABLE`, or `IRREVERSIBLE` before the user commits.

The design rule that ties them together: *an action that cannot produce a receipt cannot run.*

2 Related work

A structured review (805 deduplicated records; 38 verified works; OpenAlex and arXiv, searched October 2026) maps six literatures. We summarize the synthesis; the full review with verified citations accompanies this paper.

Tool use. Toolformer [2], ReAct [3], Gorilla [4], ToolLLM [5], and HuggingGPT [6] established invocation, with surveys consolidating the field [7, 8]. A tool call in this literature is a transient invocation: intent, authority, and data lineage are not part of the call's record. The field optimized the verb and discarded the receipt.

Agent operating systems and protocols. AIOS [9] and the LLM-as-OS vision [10] apply OS vocabulary to scheduling and context; MCP standardizes the wire format of tool calls, with its security landscape now actively mapped [11]. None standardizes a permission grant, an action record, or an undo channel.

Benchmarks. OSWorld [12], WebArena [13], τ-bench [14], WorkArena [15], and AgentDojo [16] score task completion and attack resistance. No surveyed environment measures recourse: whether state can be restored, or whether a third party can reconstruct why an action was permitted. Reversibility and auditability are unmeasured — and therefore unresearched — properties.

Permissioning and injection defense. CaMeL [17] defeats prompt injection by design with capability-style policies; MCP-Secure [18], MiniScope [19], and Permission Manifests [20] implement runtime access control, least privilege, and declared scopes. PrivacyAsst [22] guards privacy in tool use. The counterweight is sobering: adaptive attacks break deployed defenses [21]. Two consequences follow for design: permission must be a structural field checked at commit time, not an advisory classifier; and containment must be evaluated adversarially, expecting bypasses.

Reversibility. The saga pattern [23] pairs forward actions with compensating transactions; SagaLLM [24] ports transaction guarantees to multi-agent LLM planning; reversible-systems research formalizes invertibility [25]. Compensation is proven — but applied to workflow failure, not user recourse, and it assumes hand-written compensations. No surveyed system derives and stores a per-action undo recipe at execution time, or classifies irreversibility for the user before commitment.

Audit and oversight. AgentTrace [26], PROV-AGENT [27], provenance-sensitivity audits [28], and pre-action authorization [29] capture fragments: what happened, what data flowed, who consented. Human-in-the-loop systems [30, 31] add approval, itself an attack surface [32]; oversight limits are argued broadly [33]. None fuses the three into a single record sufficient to re-narrate the action without rerunning anything.

Positioning. Khatt's closest neighbours are SagaLLM (transactions for planning, no user-facing reversibility classes), CaMeL (capability defense at the model, no durable record), and PROV-AGENT (provenance of interactions, no undo). The gap Khatt fills: permission, record, and undo as one artifact in the data substrate.

3 Design

3.1 The action record

Every action, from proposal to undo, is one record in the Qafiya store. It carries: the user's verbatim intent and its normalized form; the actor (agent and session); the authority (user and grant); an embedded copy of the permission scope; machine-checked preconditions with results; the reversibility class; the undo recipe; data inputs with pre/post content hashes; the execution log; the outcome (`success`, `failed`, `refused`, `partial`); undo status; and recorded confirmations. Refusals and failures are terminal states with kept records: negative results are data, not noise.

3.2 The permission gate

Permission grants are immutable records: action types, a target pattern, constraints, and expiry. The gate checks, in order and in deterministic code: grant exists and unexpired; action type is granted; every target matches the pattern *after path canonicalization*, so traversal attempts such as `../` fail by construction; constraints hold; and actions requiring confirmation carry a recorded confirmation before execution. Any failure refuses the action with a machine-readable reason. No model judgment is consulted: the adaptive-attack evidence shows advisory filters degrade under pressure, so the enforcement core is structural.

3.3 Undo semantics

Undo recipes are executable operations captured *before* execution: source paths and content snapshots for file edits; trash identifiers for deletes (captured at the moment of the move, which is what creates them); counter-operations for calendar and draft actions. Three classes are assigned at arm time and shown to the user before commitment:

- `EXACT` — undo restores byte-for-byte state, verified by content-hash comparison.
- `COMPENSABLE` — a counter-action restores semantic state (trash restore, field restore); the undo is itself a chained record.
- `IRREVERSIBLE` — no undo exists (a sent email, a purged trash entry). These actions require an explicit recorded confirmation and carry the provider's message identifier as provenance of the external effect.

If a recipe cannot be captured, the action is classified `IRREVERSIBLE` automatically. Deletes never unlink: they move entries into a managed trash inside the Qafiya store, where trash contents are themselves provenance-carrying records, and purging the trash is a recorded irreversible action.

Undo is verified, not assumed. Conflict detection compares current content hashes against the record's post-action hashes: if the world changed since the action, undo refuses with a reason rather than silently overwriting a user's later work.

3.4 Tamper-evident log

The action log is a hash-chained history inherited from the Qafiya substrate (v1.3) [1]: every state transition of every record appends a chained entry to the dataset's `history.jsonl`, so modification or deletion of history breaks the chain and is detected by a verification pass that recomputes every hash. The mechanism is proven at the substrate and applied at the action layer. Tamper-evidence detects altered history; it does not prevent wholesale deletion of the store, which we leave to replication and backup as future work.

4 Implementation

The prototype is a Python package implementing the lifecycle state machine (PROPOSED → GATED → ARMED → EXECUTING → DONE/FAILED/REFUSED → UNDONE), the deterministic gate, and three adapters: a file executor over Qafiya stores, a calendar adapter, and an email adapter that hands delivery to an existing sending system (we integrate; we do not rebuild). The record is written at every state transition; an action whose record cannot be written does not run. The Layer-2 view renderer is mocked behind a narrow interface (`render_confirmation`, `render_audit_view`) until its frozen spec exists; the engine depends on the interface, not the mock. Code is AGPL-3.0 under the DeeWaanAI GitHub organization.

5 Evaluation

Experiments follow the Qafiya discipline: fixtures frozen before runs, and a spine script (`run_all.py`) that recomputes every headline number from raw artifacts. If the script fails, the number does not appear here. Fixtures: a frozen battery of 41 actions across fifteen action types, each executed against a fresh, identically seeded store, and 15 injected defects. All numbers below are script output.

5.1 Track A: reversibility

A framing note before the numbers. Exact reversibility of `EXACT` actions is true *by construction*: the undo recipe is captured before execution, so restoring it should always work. We therefore read the 37/37 figure as a correctness check on the implementation — a floor, not a ceiling — not as the finding. The informative results are the three behaviors the surveyed literature does not offer: reversibility is *classified and shown to the user before commitment*; undo is *refused when the world changed* after the action; and irreversible actions are *gated behind explicit recorded confirmation.*

Metric	Result
Battery size	41 actions
Undoable actions (EXACT + COMPENSABLE)	37
Restored byte-for-byte (state-hash verified)	**37/37 = 100%**
Irreversible actions correctly gating undo	2/2 (email send, trash purge)
Limit probes passed	2/2
All statuses and classes as specified	yes

Table 1: Track A: reversibility over the frozen action battery. The 37/37 figure is a by-construction correctness check; the findings are pre-commitment classification, conflict refusal, and irreversibility gating.

Undo requested twice is a recorded no-op, not an error. Undo after an external modification is detected via hash mismatch and *refused with a reason* — the system protects the user's later change instead of destroying it. Irreversible actions behave honestly: the sent email reports `undo: impossible`, and its record carries the provider message ID as the audit trail of an effect that cannot be taken back.

5.2 Track B: permission containment

Fifteen defects were injected in the style of the Qafiya integrity track: out-of-scope targets, three path-traversal variants, an expired grant, a missing grant, an unknown action type, failed preconditions (ghost files, ghost events, malformed recipients, occupied destinations), missing irreversible-action confirmations, and a two-step chain in which a legal in-scope edit is followed by a quiet out-of-scope edit.

Metric	Result
Injected defects	15
Caught before execution	**15/15 = 100%**
State change on refused actions	none

Table 2: Track B: defect-injection containment.

The two-step chain deserves note: the legal first action executed and was recorded; the out-of-scope second action was refused at the gate, and the store state reflected only the legal action. Containment is per-action and structural, not per-session and statistical.

5.3 Track C: auditability

Metric	Result
Actions fully re-narratable from record alone	**39/39**
Hash chain verified over full run	yes
Tamper by record modification detected	yes
Tamper by record deletion detected	yes

Table 3: Track C: auditability and tamper-evidence.

Re-narratability is the strong property: a complete account — intent, authority, scope, preconditions, steps, outcome, undo path — reconstructed from the record alone, with no reruns and no side logs. Two battery actions were skipped in the full-chain run because earlier actions consumed their targets; this is recorded in the results artifact, not hidden.

5.4 A worked example

A request to "fix the Q3 numbers" produces, in order: a record with the verbatim intent and its normalization (`file.edit`, `reports/q3.txt`); a gate check against the embedded grant; a precondition pass (`target_exists`); classification `EXACT`; capture of a snapshot-based undo recipe; execution; and a final record carrying pre/post content hashes. An undo request replays the recipe and the post-state hashes equal the pre-state hashes — the system does not claim the undo worked; it proves it. Contrast with the refusal path: "wipe everything" under a grant limited to `file.edit` in `reports/**` never reaches the executor, and its refusal record is itself part of the audit trail.

6 What Khatt provably cannot do

Honesty is architecture, not rhetoric. The limits, measured where possible:

1. **Irreversible actions cannot be undone.** A sent email is gone; Khatt gates it, records it, and says so before commitment — nothing more is possible, and the system claims nothing more.
2. **Undo conflicts are detected and refused, not resolved.** When the world changed after an action, automatic undo would destroy newer state. Resolution policy is future work.
3. **The gate constrains normalized actions, not intent fidelity.** If the parser misunderstands the request, the record faithfully documents a correctly-gated wrong action. Intent quality is Layer 5's problem; this paper claims enforcement and recourse, not understanding.
4. **Tamper-evidence is not tamper-proofing.** The chain detects altered or deleted history; it does not prevent destruction of the whole store.
5. **Containment is evaluated against our injection set.** Fifteen defects, adversarially chosen, is a floor, not a ceiling; the adaptive attack literature warns that static defense claims age poorly, so our harness is built to be extended.

7 AI-usage disclosure

Kimi K3 (Moonshot AI) was used as the engineering and drafting assistant for this layer: it implemented the prototype and experiment harness under the frozen specification, ran the verified literature search, and produced draft text that the author reviewed, corrected, and approved. No model made unsupervised decisions: the specification, the frozen fixtures, the three owner design decisions (email via an external sending system; managed trash inside the Qafiya store; hash-chained log from v1), and this paper's claims are the author's.

AI involvement is itself audited, not asserted. Every design decision is logged (D1–D6 in the accompanying decision log), and every number in this paper traces through a two-way claim map to a frozen artifact; the semantic map checks that the prose means what the data says. Three oversight episodes are on the record:

1. **E1.** The first spine run reported 64.9% reversibility. The defect was in the AI-written undo code (empty parent directories left behind) and in the AI-written measurement (byte-wise rather than semantic hashing of adapter state). Caught by the frozen-spine discipline before any number reached the paper; fixed; re-run to 37/37. *Lesson: headline numbers must be recomputed from raw artifacts by a script that can fail.*
2. **E2.** The draft claimed "nine action types"; the frozen battery contains fifteen — a paper-only number introduced during AI drafting. Caught by the numeric claim map and corrected. *Lesson: a number that exists only in the paper is a defect.*
3. **E3.** The draft said "two battery actions were skipped" beside a 36/36 table; the artifact said five. Root cause was an AI-written harness bug marking artifact-less setup actions as failed dependencies. Caught by the semantic pass comparing prose against frozen data; harness fixed; re-run gives two legitimate skips and 39/39. *Lesson: value fidelity is not meaning fidelity; interpretive sentences need their own evidence links.*

The enforcement artifacts — the spine script, the frozen fixtures, the two-way verifier, and both claim maps — ship with this paper, so a reviewer can re-run every check rather than take this disclosure on faith.

8 Conclusion

The agent literature learned to act. Khatt asks the preceding questions: may it act, and can it take it back? By binding permission, record, and undo recipe into one provenance-carrying artifact in the

Qafiya substrate, the action layer makes every action auditable from its own record and reversible by construction where physics allows — and honest where it does not. The frozen experiments show the design holds: 100% exact reversibility for undoable actions, 100% containment of injected permission and precondition defects, and complete re-narratability under a tamper-evident chain. The remaining layers of DeeWaanAI OS — memory, routing, and the shell — build on this record as their contract.

References

[1] Hariri, Y. (2026). Qafiya: A Provenance-First Data Model for LLM-Mediated Personal Computing (DeeWaanAI OS Layer 1). DeeWaanAI. DOI placeholder pending publication; spec v1.1 frozen, hash-chained history v1.3.

[2] Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv. https://doi.org/10.48550/arXiv.2302.04761

[3] Yao, S., Zhao, J., Yu, D., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv. https://doi.org/10.48550/arXiv.2210.03629

[4] Patil, S. G., Zhang, T., Wang, X., et al. (2023). Gorilla: Large Language Model Connected with Massive APIs. arXiv. https://doi.org/10.48550/arXiv.2305.15334

[5] Qin, Y., Liang, S., Ye, Y., et al. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv. https://doi.org/10.48550/arXiv.2307.16789

[6] Shen, Y., Song, K., Tan, X., et al. (2023). HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. arXiv. https://doi.org/10.48550/arXiv.2303.17580

[7] Qu, C., Dai, S., Wei, X., et al. (2025). Tool learning with large language models: a survey. Frontiers of Computer Science. https://doi.org/10.1007/s11704-024-40678-2

[8] Plaat, A., van Duijn, M. J., van Stein, B., et al. (2025). Agentic Large Language Models, a Survey. JAIR. https://doi.org/10.1613/jair.1.18675

[9] Mei, K., Zhu, X., Xu, W., et al. (2024). AIOS: LLM Agent Operating System. arXiv. https://doi.org/10.48550/arXiv.2403.16971

[10] Ge, Y., Ren, Y., Hua, W., et al. (2023). LLM as OS, Agents as Apps. arXiv. https://doi.org/10.48550/arXiv.2312.03815

[11] Hou, X., Zhao, Y., Wang, S., et al. (2026). Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. ACM TOSEM. https://doi.org/10.1145/3796519

[12] Xie, T., Zhang, D., Chen, J., et al. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv. https://doi.org/10.48550/arXiv.2404.07972

[13] Zhou, S., Xu, F. F., Zhu, H., et al. (2023). WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv. https://doi.org/10.48550/arXiv.2307.13854

[14] Yao, S., Shinn, N., Razavi, P., et al. (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv. https://doi.org/10.48550/arXiv.2406.12045

[15] Drouin, A., Gasse, M., Caccia, M., et al. (2024). WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? arXiv. https://doi.org/10.48550/arXiv.2403.07718

[16] Debenedetti, E., Zhang, J., Balunović, M., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv. https://doi.org/10.48550/arXiv.2406.13352

[17] Debenedetti, E., Shumailov, I., Fan, T., et al. (2025). Defeating Prompt Injections by Design (CaMeL). arXiv. https://doi.org/10.48550/arXiv.2503.18813

[18] Singh, G., Madisetti, V. K. (2026). MCP-Secure: A Runtime Access Control Layer for Privilege-Aware LLM Agent Tooling. IEEE OJCS. https://doi.org/10.1109/ojcs.2026.3664314

[19] Zhu, J., Huang, X., Tseng, K., et al. (2025). MiniScope: Authorizing Agents with Least-Privilege Permissions. arXiv. https://doi.org/10.48550/arXiv.2512.11147

[20] Marro, S., Chan, A., Ren, X., et al. (2026). Permission Manifests for Web Agents. arXiv. https://doi.org/10.48550/arXiv.2601.02371

[21] Zhan, Q., Fang, R., Panchal, H. S., et al. (2025). Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents. NAACL Findings. https://doi.org/10.18653/v1/2025.findings-naacl.395

[22] Zhang, X., Xu, H., Ba, Z., et al. (2024). PrivacyAsst: Safeguarding User Privacy in Tool-Using Large Language Model Agents. IEEE TDSC. https://doi.org/10.1109/tdsc.2024.3372777

[23] Štefanko, M., Chaloupka, O., Rossi, B. (2019). The Saga Pattern in a Reactive Microservices Environment. ICSOFT. https://doi.org/10.5220/0007918704830490

[24] Chang, E. Y., Geng, L. (2025). SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. PVLDB. https://doi.org/10.14778/3750601.3750611

[25] Mezzina, C. A., Schlatte, R., Glück, R., et al. (2020). Software and Reversible Systems: A Survey of Recent Activities. LNCS. https://doi.org/10.1007/978-3-030-47361-7_2

[26] AlSayyad, A., Huang, K. Y., Pal, R. (2026). AgentTrace: A Structured Logging Framework for Agent System Observability. arXiv. https://doi.org/10.48550/arXiv.2602.10133

[27] de Souza, R. P., Gueroudji, A., DeWitt, S. J., et al. (2025). PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows. IEEE eScience. https://doi.org/10.1109/escience65000.2025.00093

[28] Liao, J. (2026). Auditing Provenance Sensitivity in LLM Agent Action Selection. arXiv. https://doi.org/10.48550/arXiv.2607.20827

[29] Grice, D. (2026). Provenance-Based Pre-Action Authorization for LLM Agents. Zenodo. https://doi.org/10.5281/zenodo.21270299

[30] Mozannar, H., Bansal, G., Tan, C., et al. (2025). Magentic-UI: Towards Human-in-the-loop Agentic Systems. arXiv. https://doi.org/10.48550/arXiv.2507.22358

[31] Irshad, H., Mughees, A., Mughees, N., et al. (2026). The Verifiable Action Card: Trustworthy Human-in-the-Loop Control for Secure Autonomous Agents. arXiv. https://doi.org/10.48550/arXiv.2609.18411

[32] Kumar, A. A. (2026). Loopjacking: Hijacking Human-in-the-Loop Approval. arXiv. https://doi.org/10.48550/arXiv.2609.21081

[33] Holzinger, A., Zatloukal, K., Müller, H. (2024). Is human oversight to AI systems still possible? New Biotechnology. https://doi.org/10.1016/j.nbt.2024.12.003